Firewall Protocol

Documentation Technique

Serious Game de sensibilisation au phishing

Développé avec Unity 6 (6000.3.3f1)

Plateforme cible : Android

17 mars 2026

Table des matières

1 Présentation du projet **2**
1.1 Concept . . . 2
1.2 Objectifs pédagogiques . . . 2
1.3 Technologies utilisées . . . 2

2 Architecture du projet **2**
2.1 Organisation des fichiers . . . 2
2.2 Design Patterns utilisés . . . 3
2.2.1 Singleton . . . 3
2.2.2 Observer (callbacks) . . . 3

3 Scripts principaux **4**
3.1 GameManager.cs — Logique de jeu . . . 4
3.1.1 Gestion du bouclier . . . 4
3.2 PlayerProgress.cs — Progression et sauvegarde . . . 5
3.2.1 Système de niveaux (DayConfig) . . . 5
3.3 EmailLoader.cs — Chargement des emails . . . 5
3.4 AudioManager.cs — Gestion audio . . . 5
3.5 LeaderboardManager.cs — Classement en ligne . . . 6
3.6 LoadingScreen.cs — Écran de chargement . . . 6

4 Interface utilisateur **7**
4.1 Navigation entre les écrans . . . 7
4.2 MainMenu.cs — Menu principal . . . 7
4.3 EmailCardUI.cs — Carte email . . . 7
4.4 ShopUI.cs — Boutique . . . 7
4.5 LeaderboardUI.cs — Classement . . . 7
4.6 PauseMenu.cs — Menu pause . . . 7

5 Effets visuels et sonores **8**
5.1 Effets visuels . . . 8
5.2 Musiques . . . 8
5.3 Effets sonores . . . 8

6 Données **8**
6.1 Structure d'un email (EmailData) . . . 8
6.2 Fichier JSON (emails.json) . . . 9
6.3 Sauvegarde locale (PlayerPrefs) . . . 9

7 Déploiement **9**
7.1 Build Android . . . 9
7.2 Prérequis . . . 9
7.3 Évolutions prévues . . . 10

8 Diagramme de classes simplifié **10**

1 Présentation du projet

1.1 Concept

Firewall Protocol est un serious game mobile développé sous Unity, dont l'objectif est de sensibiliser les joueurs à la détection du phishing. Le joueur incarne un employé qui doit trier des emails en les approuvant ou rejetant via un système de swipe (glisser gauche/droite).

1.2 Objectifs pédagogiques

- Apprendre à identifier les emails frauduleux (phishing)
- Reconnaître les indices visuels et textuels d'une tentative de phishing
- Développer un réflexe critique face aux emails suspects
- Sensibiliser un large public (tous âges) aux risques du phishing

1.3 Technologies utilisées

Technologie	Usage
Unity 6 (6000.3.3f1)	Moteur de jeu
C#	Langage de programmation
Firebase Firestore	Base de données en ligne (classement)
Firebase Auth	Authentification anonyme
DOTween	Bibliothèque d'animations
TextMeshPro	Rendu de texte avancé
JSON	Format de stockage des emails
PlayerPrefs	Sauvegarde locale de la progression

2 Architecture du projet

2.1 Organisation des fichiers

Le projet suit une architecture modulaire avec séparation des responsabilités :

```
Assets/
+-- Scripts/
|   +-- Managers/          # Logique centrale
|   |   +-- GameManager.cs
|   |   +-- AudioManager.cs
|   |   +-- PlayerProgress.cs
|   |   +-- LeaderboardManager.cs
|   |   +-- EffectsManager.cs
|   |   +-- ShopSystem.cs
|   +-- Data/              # Modeles de donnees
|   |   +-- EmailData.cs
|   |   +-- EmailJSON.cs
```

```
|   |   +-- EmailLoader.cs
|   |   +-- LeaderboardEntry.cs
|   +-- UI/                # Interface utilisateur
|   |   +-- MainMenu.cs
|   |   +-- EmailCardUI.cs
|   |   +-- ShopUI.cs
|   |   +-- LeaderboardUI.cs
|   |   +-- PauseMenu.cs
|   |   +-- LoadingScreen.cs
|   |   +-- FeedbackPopup.cs
|   |   +-- StreakUI.cs
|   |   +-- PowerUpBarUI.cs
|   |   +-- EndScreenAnimator.cs
|   +-- Environment/       # Elements du decor
|       +-- ApartmentScreen.cs
|       +-- ApartmentZone.cs
+-- Audio/
|   +-- Music/             # Musiques de fond
|   +-- SFX/               # Effets sonores
+-- Sprites/               # Images et sprites
+-- Resources/
|   +-- emails.json        # Base de donnees des emails
+-- Scenes/
    +-- SampleScene.unity  # Scene principale
```

2.2 Design Patterns utilisés

2.2.1 Singleton

La plupart des managers utilisent le pattern Singleton pour garantir une instance unique accessible globalement :

```csharp
public static GameManager Instance;

void Awake()
{
    if (Instance == null)
        Instance = this;
    else
        Destroy(gameObject);
}
```

Classes utilisant ce pattern : `GameManager`, `AudioManager`, `PlayerProgress`, `LeaderboardManager`, `EffectsManager`, `MainMenu`, `EmailLoader`.

2.2.2 Observer (callbacks)

La communication avec Firebase utilise des callbacks asynchrones :

```
public void GetTopScores(int limit, Action<List<LeaderboardEntry>>
    callback)
{
    db.Collection(COLLECTION_NAME)
        .OrderByDescending("bestScore")
        .Limit(limit)
        .GetSnapshotAsync()
        .ContinueWithOnMainThread(task =>
        {
            // ... traitement ...
            callback?.Invoke(entries);
        });
}
```

3 Scripts principaux

3.1 GameManager.cs — Logique de jeu

Le `GameManager` est le chef d'orchestre du jeu. Il gère :

- L'initialisation des parties (chargement des emails selon le jour)
- Le traitement des décisions du joueur (approuver/rejeter un email)
- Le calcul du score, des coins et de l'intégrité
- La gestion des power-ups (bouclier, indice, skip)
- Les conditions de victoire et de game over
- La navigation entre les écrans (appartement, menu)

Flux de jeu principal

1. `InitialiserPartie()` : charge les emails du jour, initialise les stats
2. `ChargerEmailSuivant()` : affiche l'email suivant
3. `TraiterDecisionAvecRetour()` : évalue la réponse du joueur
4. Si correct : +score, +coins, +streak, email suivant
5. Si incorrect : -intégrité, feedback explicatif, vérification game over
6. `FinDeJournee()` : calcul des bonus, sauvegarde, soumission au leaderboard

3.1.1 Gestion du bouclier

Quand le joueur se trompe et possède un bouclier actif, les dégâts sont réduits de 50% et le bouclier est consommé. Le SFX joué est celui du shield break (et non le SFX d'erreur classique) pour éviter la superposition de sons :

```
if (shieldUsed)
    AudioManager.Instance.PlayShieldBreak();
else
    AudioManager.Instance.PlayWrong();
```

3.2 PlayerProgress.cs — Progression et sauvegarde

Gère toute la progression du joueur avec sauvegarde automatique via `PlayerPrefs` au format JSON.

- **Progression** : jour actuel, plus haut jour atteint, coins
- **Statistiques** : emails traités, bonnes/mauvaises réponses, parties jouées
- **Boutique** : items possédés, vies, indices, skips
- **Streak** : série de bonnes réponses consécutives

3.2.1 Système de niveaux (DayConfig)

Chaque jour a une configuration dynamique calculée par `GetDayConfig(int day)` :

Jours	Emails	Intégrité	Répartition
1–2	6–7	100%	100% facile
3–4	8–9	90–95%	60% facile, 40% moyen
5–6	10–11	80–85%	30% facile, 50% moyen, 20% difficile
7–8	12–13	70–75%	10% facile, 40% moyen, 40% difficile, 10% expert
9+	14–15	60%	5% facile, 25% moyen, 40% difficile, 30% expert

Le multiplicateur de récompense augmente de 20% par jour (1×, 1.2×, 1.4×...).

3.3 EmailLoader.cs — Chargement des emails

Charge la base de données d'emails depuis un fichier JSON dans `Resources/`. Sélectionne les emails pour chaque journée selon la difficulté configurée, en utilisant l'algorithme de mélange Fisher-Yates.

Processus de sélection des emails

1. Récupère la configuration du jour (`DayConfig`)
2. Filtre les emails par difficulté (facile, moyen, difficile, expert)
3. Mélange chaque groupe aléatoirement
4. Sélectionne le nombre requis de chaque difficulté
5. Mélange final pour éviter le regroupement par difficulté

3.4 AudioManager.cs — Gestion audio

Singleton gérant la musique de fond et les effets sonores avec deux `AudioSource` séparés.

- **Musiques** : menu, gameplay, appartement — avec crossfade via DOTween

— **SFX** : bonne/mauvaise réponse, shield break, streak break, swipe, achat, victoire, game over
— **Volume** : contrôle via slider dans le menu pause, sauvegardé dans PlayerPrefs
— **Mute** : toggle avec changement d'icône (on/off)

Certains SFX ont des multiplicateurs de volume personnalisés :

— Swipe email : ×4 (pour être audible pendant le gameplay)
— Shield break : ×3 (pour se démarquer)
— Streak break : ×7 (pour l'impact)
— Skip : ×100 avec pitch ×3 (effet accéléré)

Le crossfade entre musiques utilise DOTween :

```
musicSource.DOFade(0f, musicFadeDuration).OnComplete(() =>
{
    musicSource.clip = clip;
    musicSource.loop = true;
    musicSource.Play();
    musicSource.DOFade(isMuted ? 0f : musicVolume,
       musicFadeDuration);
});
```

3.5 LeaderboardManager.cs — Classement en ligne

Gère la communication avec Firebase Firestore pour le classement.

— **SubmitScore()** : envoie le score si meilleur que l'existant
— **GetTopScores()** : récupère le top 8 trié par score décroissant
— **GetPlayerRank()** : calcule le rang du joueur
— **IsPseudoTaken()** : vérifie l'unicité du pseudo avant inscription

Structure d'un document Firestore :

```
leaderboard/{pseudo}
+-- pseudo: string
+-- bestScore: int
+-- highestDay: int
+-- timestamp: ServerTimestamp
```

3.6 LoadingScreen.cs — Écran de chargement

Affiche un écran de chargement animé au lancement du jeu :

— Fond noir avec la boîte aux lettres au centre
— Enveloppes animées qui tombent vers la boîte (spawn en boucle avec DOTween)
— Texte "Chargement..." avec animation des points
— Disparition en fade-out après un temps minimum (3 secondes)
— La musique du menu ne démarre qu'après la fin du chargement

4 Interface utilisateur

4.1 Navigation entre les écrans

```
Loading Screen --> Menu Principal --> Appartement --> Gameplay
                        |                  |              |
                        |                  |       Victoire/Game Over
                        |                  |              |
                        +-----------------+--------------+
                        (retour au menu ou a l'appartement)
```

4.2 MainMenu.cs — Menu principal

- Affiche le pseudo connecté, le jour actuel et les cryptos
- Bouton "Continuer" : reprend la partie (→ appartement)
- Bouton "Nouvelle partie" / "Recommencer" : repart du jour 1
- Bouton "Classement" : ouvre le leaderboard (demande un pseudo si pas encore défini)
- Vérification de l'unicité du pseudo via Firebase avant validation
- Animations d'entrée : fade-in du canvas, scale du logo, apparition séquentielle des boutons

4.3 EmailCardUI.cs — Carte email

Interface de swipe pour approuver/rejeter les emails. Le joueur glisse la carte à gauche (rejeter) ou à droite (approuver). Un seuil de distance est nécessaire pour valider l'action.

4.4 ShopUI.cs — Boutique

Permet d'acheter des power-ups avec les cryptos gagnés :

- Bouclier : réduit les dégâts de 50% (consommable)
- Indice : révèle si l'email est frauduleux ou légitime
- Skip : passe un email sans pénalité
- Vie supplémentaire : restaure 25% d'intégrité en cas de game over

4.5 LeaderboardUI.cs — Classement

Affiche le top 8 des joueurs avec pseudo, score et jour maximum. Le texte "Chargement..." s'affiche pendant la requête Firebase. Le rang du joueur est affiché en haut du panneau.

4.6 PauseMenu.cs — Menu pause

- Bouton reprendre (ferme le menu)
- Bouton mute (toggle son on/off avec changement d'icône)
- Bouton quitter
- Slider de volume connecté à l'AudioManager

5 Effets visuels et sonores

5.1 Effets visuels

- **GlitchEffect** : effet de glitch visuel sur mauvaise réponse (probabilité 40%) et game over (version intense)
- **ConfettiEffect** : confettis sur victoire
- **ShieldBreakEffect** : animation visuelle de destruction du bouclier
- **FloatingTextEffect** : texte flottant "+score" ou "-dégâts"
- **StreakUI** : affichage de la série de bonnes réponses
- **EndScreenAnimator** : animation des écrans de fin (victoire/game over)

5.2 Musiques

Trois ambiances musicales avec transitions en crossfade :

- Menu principal : ambiance calme d'accueil
- Gameplay : musique dynamique pendant le tri des emails
- Appartement : ambiance détendue entre les journées

5.3 Effets sonores

Événement	**SFX**
Bonne réponse	Validation positive
Mauvaise réponse	Buzzer d'erreur
Shield break	Son de bouclier brisé
Streak break	Son de série perdue (volume ×7)
Swipe email	Son de glissement (volume ×4)
Achat boutique	Son de caisse
Victoire	Fanfare
Game over	Son de défaite

6 Données

6.1 Structure d'un email (EmailData)

```
public class EmailData
{
    public string expediteurNom;       // Nom de l'exp diteur
    public string expediteurEmail;     // Adresse email
    public string objet;              // Objet du mail
    public string corpsDuMessage;     // Corps du message
    public bool estFrauduleux;        // Phishing ou non
    public string explicationErreur; // Explication pedagogique
    public int pointsSiCorrect;       // Points gagnes
    public int degatsIntegrite;       // Degats si erreur
}
```

6.2 Fichier JSON (emails.json)

Les emails sont stockés dans `Resources/emails.json` avec 4 niveaux de difficulté :

- **Facile** : indices évidents (fautes grossières, adresses suspectes)
- **Moyen** : indices plus subtils (urgence artificielle, liens trompeurs)
- **Difficile** : très réaliste, peu d'indices
- **Expert** : quasiment indiscernable d'un vrai email

6.3 Sauvegarde locale (PlayerPrefs)

La progression est sérialisée en JSON et stockée via `PlayerPrefs` :

- Clé `PlayerProgress` : toutes les données de progression
- Clé `MusicVolume` / `SfxVolume` : volumes audio
- Clé `IsMuted` : état du mute
- Clé `PlayerPseudo` : pseudo du joueur

7 Déploiement

7.1 Build Android

- Plateforme : Android (APK)
- Target API Level : 33+ (Android 13)
- Minimum API Level : 22 (Android 5.1)
- Rendering : Universal Render Pipeline (URP)
- Canvas Scaler : Scale With Screen Size, référence 1080×1920, match 0.5

7.2 Prérequis

- Unity 6 avec module Android Build Support
- Android SDK & NDK Tools
- OpenJDK (installé avec Unity)
- Compte Google Play (25$ one-time) pour la distribution

7.3 Évolutions prévues

- Migration de la base d'emails sur Firebase pour mise à jour en temps réel sans republier l'APK
- Ajout de nouveaux emails basés sur les arnaques actuelles
- Version B2B avec emails personnalisables par entreprise

8 Diagramme de classes simplifié

```
+-------------------+     +-----------------+
|   GameManager     |---->|  EmailLoader    |
|                   |     |  - emails.json  |
| - integrite       |     +-----------------+
| - score           |
| - coins           |     +-----------------+
|                   |---->| PlayerProgress  |
| TraiterDecision() |     | - currentDay    |
| FinDeJournee()    |     | - coins         |
| GameOver()        |     | - ownedItems    |
+-------------------+     | Save() / Load() |
        |                  +-----------------+
        |
        v                  +-----------------+
+-------------------+     |  AudioManager   |
|   EmailCardUI     |     | - musicSource   |
|                   |     | - sfxSource     |
| Swipe gauche/     |     | PlayMusic()     |
| droite            |     | PlaySfx()       |
+-------------------+     +-----------------+

+-------------------+     +-----------------+
| LeaderboardMgr    |---->|  Firebase       |
|                   |     |  Firestore      |
| SubmitScore()     |     +-----------------+
| GetTopScores()    |
| IsPseudoTaken()   |     +-----------------+
+-------------------+     |   ShopSystem    |
                          | - items[]       |
+-------------------+     | Purchase()      |
|  EffectsManager   |     +-----------------+
| - GlitchEffect    |
| - ConfettiEffect  |
| - ShieldBreak     |
+-------------------+
```